Machine Learning Lab – 20 Mark Answers

Experiment 1: Decision Tree Classification

Objective

To implement and evaluate a Decision Tree classifier using the Iris dataset and analyze its performance using accuracy, confusion matrix, classification report, and misclassified instances.

(a) Load, Pre-process, and Split the Dataset

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Load Iris dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['target'] = iris.target

# Display first 5 rows
print("First 5 rows:")
print(df.head())

# Display data types
print("\nData Types:")
print(df.dtypes)

# Check missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Split input and output
X = df.iloc[:, :-1]
y = df['target']
```

```
# Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("\nTraining samples:", len(X_train))
print("Testing samples:", len(X_test))
```

Explanation

- The **Iris dataset** contains 150 samples.
- It has four input features:
    1. Sepal Length
    2. Sepal Width
    3. Petal Length
    4. Petal Width
- The target contains three classes: Setosa, Versicolor, and Virginica.
- Missing values are checked using isnull().sum().
- Since the Iris features are numerical, **encoding is not required**.
- The dataset is divided into:
    - **80% Training Data**
    - **20% Testing Data**

(b) Build the Decision Tree Model

```
from sklearn.tree import DecisionTreeClassifier, export_text

# Create Decision Tree using Gini Index
model = DecisionTreeClassifier(
    criterion='gini',
    random_state=42,
```

```
    max_depth=3
)

# Train the model
model.fit(X_train, y_train)

# Print tree structure
tree_rules = export_text(
    model,
    feature_names=list(X.columns)
)

print("\nDecision Tree Structure:")
print(tree_rules)

# Root node
print("Root Node:", X.columns[model.tree_.feature[0]])
```

Tree Splitting Criterion

Here, the model uses the **Gini Index**:

$$
Gini = 1 - \sum_{i=1}^{n}p_i^2
$$

The Decision Tree selects the feature that produces the **maximum reduction in Gini impurity**.

Alternatively, Information Gain can be used:

```
model = DecisionTreeClassifier(
    criterion='entropy',
    random_state=42
)
```

Root Node and Justification

For the Iris dataset, the root node is commonly:

Petal Length (cm)

or, depending on the train-test split:

Petal Width (cm).

The exact root feature should be taken from:

X.columns[model.tree_.feature[0]]

Justification

The root node is selected because it provides the **best separation among the three classes** and produces the **maximum decrease in impurity**. Petal measurements are highly effective in separating Setosa from the other Iris species.

(c) Model Evaluation

```
from sklearn.metrics import accuracy_score

from sklearn.metrics import confusion_matrix

from sklearn.metrics import classification_report


# Predict test data

y_pred = model.predict(X_test)


# Accuracy

accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)


# Confusion Matrix

cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")

print(cm)


# Classification Report

print("\nClassification Report:")

print(classification_report(

    y_test,

    y_pred,

    target_names=iris.target_names

))
```

Find Misclassified Instances

```
results = X_test.copy()
```

```
results['Actual'] = y_test.values
results['Predicted'] = y_pred

misclassified = results[
    results['Actual'] != results['Predicted']
]

print("\nMisclassified Instances:")
print(misclassified.head(2))
```

Performance Comment

The Decision Tree generally achieves **high accuracy on the Iris dataset**, often around **90%–100%**, depending on the train-test split and tree parameters.

- **Setosa** is usually classified with very high accuracy.
- Some confusion may occur between **Versicolor** and **Virginica** because their feature values overlap.
- The confusion matrix shows the number of correct and incorrect predictions.
- Precision measures how many predicted instances are correct.
- Recall measures how many actual instances are correctly identified.
- F1-score gives the balance between precision and recall.

Misclassified Instances

At least two incorrectly predicted samples can be obtained automatically using:

```
print(misclassified.head(2))
```

Conclusion: The Decision Tree classifier performs well because the Iris dataset contains features that clearly distinguish most classes. Misclassifications mainly occur between similar classes such as Versicolor and Virginica.

Experiment 2: Reinforcement Learning – Q-Learning

Objective

To implement the Q-Learning algorithm in a **4 × 4 Grid World**, train an agent for at least 100 episodes, observe the evolution of Q-values, extract the learned policy, and analyze convergence using cumulative rewards.

(a) Define the Environment and Initialize Q-Table

Environment

The Grid World contains:

- Grid size: **4 × 4**
- Actions:
    - Up
    - Down
    - Left
    - Right
- Goal reward: **+10**
- Normal step reward: **−1**
- Obstacle reward: **−5**

Let:

- Start State = (0, 0)
- Goal State = (3, 3)
- Obstacle = (1, 1)

Python Program

```
import numpy as np
import random
import matplotlib.pyplot as plt

# Grid size
ROWS = 4
COLS = 4

# States
START = (0, 0)
GOAL = (3, 3)
OBSTACLES = [(1, 1)]

# Actions
# 0 = Up, 1 = Down, 2 = Left, 3 = Right
actions = ['Up', 'Down', 'Left', 'Right']
```

```
# Hyperparameters
alpha = 0.1       # Learning rate
gamma = 0.9       # Discount factor
epsilon = 0.2     # Exploration rate

# Initialize Q-table with zeros
Q = np.zeros((ROWS, COLS, 4))

print("Initial Q-Table:")
print(Q)
```

Hyperparameter Explanation

1. Learning Rate (\alpha = 0.1)

It controls how much newly learned information changes the existing Q-value.

2. Discount Factor (\gamma = 0.9)

It determines the importance of future rewards.

3. Exploration Rate (\epsilon = 0.2)

The agent chooses a random action with probability 0.2 to explore the environment.

Reward Function and Next-State Function

```
def get_next_state(state, action):

    row, col = state

    if action == 0:       # Up
        row -= 1
    elif action == 1:     # Down
        row += 1
    elif action == 2:     # Left
        col -= 1
    elif action == 3:     # Right
        col += 1
```

```
    # Keep agent inside grid
    row = max(0, min(ROWS - 1, row))
    col = max(0, min(COLS - 1, col))

    return (row, col)


def get_reward(state):

    if state == GOAL:
        return 10

    elif state in OBSTACLES:
        return -5

    else:
        return -1
```

(b) Run Q-Learning for 100 Episodes

The Q-Learning update equation is:

$$
Q(s,a)=Q(s,a)+\alpha
\left[
r+\gamma\max_aQ(s',a)-Q(s,a)
\right]
$$

Where:

- $Q(s,a)$ = Current Q-value
- α = Learning rate
- r = Reward
- γ = Discount factor
- s' = Next state

Q-Learning Program

```
episodes = 100
episode_rewards = []

# Representative states
state1 = (0, 0)
state2 = (2, 2)

# Dictionary to store Q-values
recorded_qvalues = {}

for episode in range(1, episodes + 1):

    state = START
    total_reward = 0
    done = False
    steps = 0

    while not done and steps < 100:

        # Epsilon-greedy action selection
        if random.uniform(0, 1) < epsilon:
            action = random.randint(0, 3)
        else:
            action = np.argmax(Q[state[0], state[1]])

        # Find next state
        next_state = get_next_state(state, action)

        # Get reward
        reward = get_reward(next_state)

        # Best future Q-value
```

```
        best_next_q = np.max(
            Q[next_state[0], next_state[1]]
        )

        # Q-Learning update
        Q[state[0], state[1], action] = (
            Q[state[0], state[1], action]
            + alpha * (
                reward + gamma * best_next_q
                - Q[state[0], state[1], action]
            )
        )

        total_reward += reward

        # Move to next state
        state = next_state
        steps += 1

        # Stop when goal is reached
        if state == GOAL:
            done = True

    episode_rewards.append(total_reward)

    # Record Q-values
    if episode in [1, 50, 100]:

        recorded_qvalues[episode] = {
            'State (0,0)': Q[0, 0].copy(),
            'State (2,2)': Q[2, 2].copy()
        }
```

```
# Display Q-values
for ep, values in recorded_qvalues.items():

    print("\nEpisode:", ep)

    print("Q-values for State (0,0):")
    print(values['State (0,0)'])

    print("Q-values for State (2,2):")
    print(values['State (2,2)'])
```

Q-Table Evolution

The program records Q-values at:

- **Episode 1**
- **Episode 50**
- **Episode 100**

The output will be different slightly each time because the agent performs random exploration.

Expected Observation

Episode	State	Observation
1	(0,0)	Most Q-values are close to zero or negative
1	(2,2)	Limited learning has occurred
50	(0,0)	Good actions start receiving higher Q-values
50	(2,2)	Values move toward the goal reward
100	(0,0)	Best action has the highest Q-value
100	(2,2)	Optimal direction becomes clear

Explanation

Initially, all Q-values are:

[
Q(s,a)=0
]

After repeated interaction with the environment:

- Actions leading toward the goal receive larger Q-values.
- Actions hitting obstacles receive lower Q-values.
- Actions taking unnecessary paths receive negative values because of the −1 step penalty.
- The Q-table gradually learns which action produces the maximum long-term reward.

(c) Extract Final Learned Policy

```
# Symbols for actions
policy_symbols = {
    0: '↑',
    1: '↓',
    2: '←',
    3: '→'
}

print("\nFinal Learned Policy:")

for i in range(ROWS):

    row_policy = []

    for j in range(COLS):

        state = (i, j)

        if state == GOAL:
            row_policy.append('G')

        elif state in OBSTACLES:
            row_policy.append('X')

        else:
            best_action = np.argmax(Q[i, j])
```

```
        row_policy.append(
            policy_symbols[best_action]
        )

    print(row_policy)
```

Example Final Policy

A typical learned policy may look like:

State	Policy
(0,0)	→	→	↓	↓
(1,0)	↓	X	→	↓
(2,0)	→	→	→	↓
(3,0)	→	→	→	G

Where:

- ↑ = Up
- ↓ = Down
- ← = Left
- → = Right
- **X** = Obstacle
- **G** = Goal

The exact policy can vary because of random exploration, but the agent should learn a path that reaches the goal while avoiding the obstacle.

Plot Cumulative Reward per Episode

```
# Calculate cumulative reward
cumulative_rewards = np.cumsum(episode_rewards)


# Plot graph
plt.figure(figsize=(8, 5))


plt.plot(
    range(1, episodes + 1),
    cumulative_rewards
)
```

```
plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Q-Learning: Cumulative Reward per Episode")
plt.grid()

plt.show()
```

Convergence Behaviour

At the beginning:

- The agent explores randomly.
- It takes inefficient paths.
- It may hit obstacles.
- Therefore, rewards are generally low.

After several episodes:

- The Q-values of good actions increase.
- The agent identifies actions leading toward the goal.
- It avoids obstacle states.
- The reward improves.

By around Episodes **50–100**, the agent should show more stable behaviour because the Q-values have started converging toward useful estimates of the optimal action values.

Justification

The Q-Learning algorithm converges because it repeatedly updates the Q-values based on immediate rewards and estimated future rewards:

[
Q(s,a) \leftarrow Q(s,a)+\alpha
[r+\gamma\max Q(s',a)-Q(s,a)]
]

Thus, actions that produce a higher long-term reward eventually receive higher Q-values. The final policy selects:

[
\pi(s)=\arg\max_a Q(s,a)
]

Therefore, the agent learns an **optimal or near-optimal path** from the start state to the goal.

Final Conclusion

Experiment 1 – Decision Tree

The Decision Tree classifier successfully classified the dataset using the **Gini Index**. The model was evaluated using accuracy, confusion matrix, and classification report. The root node was selected based on the feature that produced the greatest impurity reduction. The model achieved high performance, with only a small number of possible misclassifications.

Experiment 2 – Q-Learning

The Q-Learning agent successfully learned through trial and error in the 4×4 Grid World. Initially, the Q-table contained only zeros, but after 100 episodes, useful Q-values developed for actions leading toward the goal. The final learned policy selected the action with the maximum Q-value at each state, demonstrating reinforcement learning and convergence toward an optimal path.